Systemic Resilience Copilot — Technical Report

Hack Apertus Online 2026
Track: **2B — Apertus Adoption: Own Project**

1. Problem

Banking ICT-resilience evidence often arrives as prose: architecture notes, recovery-test descriptions, continuity assumptions and partial statements about shared recovery resources. The difficult decision is not merely whether a backup exists, but **which recovery condition fails first, what remains unknown, and what action is justified next**.

Systemic Resilience Copilot turns that narrative evidence into a transparent decision path while deliberately refusing unsupported downstream conclusions.

2. What the prototype does

The production flow is:

Scenario → Apertus → validated evidence → deterministic rules → decision → Apertus challenge/explanation → fix / next evidence gap → systemic lens

Apertus parses a normal-language recovery scenario into a constrained schema. A host-side validation layer preserves explicit facts and keeps missing facts as `UNKNOWN`. Deterministic rules then identify the first binding or unresolved condition. Apertus may challenge and explain that fixed result, but cannot override it.

The main synthetic demo uses a 60-minute recovery tolerance and an observed recovery time of 95 minutes. Once recovery-path independence and execution testing are established, the rule engine stops at **S5 RED / B2 / REPAIR_EXECUTION**. After a narrow execution fix reduces recovery to 45 minutes, the chain reruns and exposes **S6 UNKNOWN** rather than declaring full resilience. A separate synthetic systemic case then reaches **S7 UNKNOWN** because shared recovery capacity, priority and coordination are not established.

3. Why Apertus is purposeful

Apertus is used where language understanding adds value:

1. **Before the decision** — parse unstructured scenario text into explicit fields.
2. **After the decision** — challenge the interpretation, explain the result and surface missing evidence.

The model is intentionally not used for the state transition itself. This separation avoids turning model confidence into a hidden risk score.

AI / rule responsibility split

Apertus may:

- extract explicit facts from narrative text;
- preserve missing evidence as `UNKNOWN`;
- challenge and explain a deterministic result;
- identify which supplied evidence is still missing.

Apertus may not:

- invent a provider failure;
- infer a systemic event probability;
- declare a capacity shortage without measured demand/capacity evidence;
- override the deterministic state or action.

4. Technical architecture

Front end

- `app-v10.html`
- `app-v10.css`
- `app-v10.js`

Apertus API layer

- `api/apertus/status.js`
- `api/apertus/parse.js`
- `api/apertus/explain.js`
- `api/apertus/smoke.js`

Validation / contracts

- `lib/scenario-parser.js`
- `lib/apertus-contract.js`

Model

- `swiss-ai/Apertus-8B-Instruct-2509:publicai`

The live hackathon deployment currently reaches Apertus through an OpenAI-compatible inference route. `APERTUS_API_BASE` is configurable, so the product logic is not tied to one hosted provider.

5. Explicit-fact reconciliation

A live-model extraction can occasionally omit a fact that is literally present in the submitted scenario. To make the decision path robust without adding inference, the host performs a narrow reconciliation step after Apertus extraction.

The reconciliation layer may only restore facts directly evidenced by the text, for example:

- a named workload;
- an explicitly stated tolerance;
- an explicitly stated recovery time;
- an explicit statement that the recovery route is outside the primary failure domain;
- an explicit statement that the same workload is supported and tested.

It cannot infer adequacy, probability, provider danger, shared failure or capacity shortage.

6. Deterministic decision logic

The rule engine follows an ordered recovery chain and stops at the first `RED` or unresolved `UNKNOWN` state.

Key guardrails:

- shared provider ≠ shared failure;
- provider diversity ≠ demonstrated independent recoverability;
- backup existence ≠ executable recovery;
- downstream capacity conclusions are blocked until upstream independence and execution pass;
- `UNKNOWN` remains `UNKNOWN`;
- no probability or provider-danger score is produced.

The main demo therefore yields:

`95 min > 60 min → S5 RED / B2 → REPAIR_EXECUTION`

After the synthetic execution fix:

`45 min < 60 min → S5 passes → S6 UNKNOWN`

Systemic supporting case:

`S4 PASS + S5 PASS + 3 simultaneous synthetic claimants + assured capacity/rights UNKNOWN → S7 UNKNOWN / MEASURE_CAPACITY`

7. Public-evidence boundary

The prototype is public-safe and reproducible, but it is not presented as equivalent to supervisory information.

Public evidence may support disclosed service relationships, workload context, stated recovery-path facts, public incidents and disclosed test results. Bank-specific architecture, identity dependencies, contractual entitlements, assured recovery capacity, priority rights and the complete simultaneous-demand set may require private or supervisory evidence.

Product rule: evidence stops → inference stops.

8. Production proof

The public deployment includes an automated production smoke endpoint and CI gate.

Latest closure proof recorded during the hackathon:

- mode: `live`;
- model: `swiss-ai/Apertus-8B-Instruct-2509:publicai`;
- critical workload recovered from the canonical scenario;
- target: 60 minutes;
- failure-domain state: YES;
- recovery route outside failure domain: YES;
- same workload supported: YES;
- execution tested: YES;
- observed recovery: 95 minutes;
- simultaneous demand: UNKNOWN;
- capacity: UNKNOWN;
- deterministic expected result: **S5 RED / B2 / REPAIR_EXECUTION**;
- secret exposure: false.

One production smoke run observed a parse latency of 4.321 seconds. This is a single operational check, not a performance benchmark.

9. Technical rigour

The prototype uses:

- constrained JSON contracts;
- enum and numeric validation;
- host-side explicit-fact reconciliation;
- strict release classes for public/synthetic inputs;
- deterministic decision rules;
- server-side secret handling;
- safe fallback behavior;
- automated local validation;
- automated production smoke testing;
- public repository documentation of assumptions and limits.

10. Value, cost and scalability

The design keeps LLM usage narrow:

- one parse call for the scenario;
- one optional explanation/challenge call;
- deterministic rule evaluation runs locally and does not require an LLM call.

This reduces model dependence and makes batch processing feasible. Actual inference cost depends on the deployment provider and is not claimed in this prototype.

The product can scale by separating:

1. ingestion/parsing;
2. validated structured evidence;
3. deterministic rule evaluation;
4. optional human-facing explanation.

11. Sovereign deployability

The architecture is designed around an open Apertus model and a configurable inference endpoint. The current public demo uses hosted inference for accessibility during the hackathon. A private or sovereign deployment can point the adapter to an alternative compatible Apertus endpoint while preserving the same rule engine and UI.

A fully air-gapped deployment is **not demonstrated** in this prototype; it would additionally require packaging and operating a local/on-prem Apertus inference server.

12. Implementation feasibility

The prototype is already deployed and testable in a browser. The core flow is implemented end-to-end:

- narrative scenario input;
- live Apertus parse;
- schema validation;
- deterministic decision;
- Apertus explanation/challenge;
- narrow intervention and rerun;
- systemic evidence gate;
- production smoke validation.

The design deliberately avoids a broad predictive model. It focuses on a small, explainable workflow that can be integrated into existing resilience review processes.

13. What was new during Hack Apertus

The **Systemic Resilience Copilot product implementation was created during the Hack Apertus build period**. The public repository contains the hackathon implementation: UI, Apertus adapter, constrained parse/explanation contracts, deterministic workflow implementation, public-safe synthetic cases, systemic lens, evidence UX, production smoke gate and demo materials.

The project was informed by pre-existing research on banking ICT recoverability. That research remains separate background IP and is not published in this repository. No unpublished manuscript text, private research files, reviewer-sensitive material, confidential bank data or private repository contents are included in the hackathon output.

14. Data

No external dataset is required for the main submission. The demo uses synthetic, public-safe scenarios created for the prototype. Therefore a separate dataset submission is not applicable to this project.

15. Limitations

The prototype does not claim:

- predictive accuracy for real incidents;
- a probability of systemic failure;
- a provider danger score;
- that shared exposure proves shared failure;
- that shared recovery resources prove a capacity shortage;
- equivalence between public evidence and supervisory/private information.

16. Reproducibility

Repository validation:

```
npm run validate
```

Mock local demo:

```
npm run dev:mock
```

The public repository contains the source, contracts, deployment configuration, safety boundaries and smoke-test workflow required to inspect the prototype.

17. Links

- Live demo: https://systemic-resilience-copilot.vercel.app/
- Source: https://github.com/rostys-fintech/systemic-resilience-copilot

18. Licensing

Hackathon source code is released under **Apache License 2.0**. Hackathon documentation, designs and text are released under **CC BY 4.0**. No submitted dataset is included; if a dataset is added later for the hackathon, it must follow the event's required dataset licence and provenance rules.